

Towards a Lightweight CRYSTALS-Kyber in FPGAs: an Ultra-lightweight BRAM-free NTT Core

Ziying Ni*, Ayesha Khalid*, Weiqiang Liu[†] and Máire O'Neill*

Email: {zni03,a.khalid}@qub.ac.uk, liuweiqiang@nuaa.edu.cn, m.oneill@ecit.qub.ac.uk.

* Centre for Secure Information Technologies (CSIT), Queen's University Belfast, Belfast, UK

[†]College of Electronics and Information Engineering, Nanjing University of Aeronautics and Astronautics, Nanjing, China

Abstract—CRYSTALS-Kyber is the first quantum-resilient, lattice-based Public Key Encryption (PKE)/Key Encapsulation Mechanism (KEM) cryptosystem that is chosen by the ongoing National Institute of Standards and Technology post-quantum cryptography standardization (NIST PQC) for standardization. This work presents a lightweight and efficient, FPGA-based hardware implementation for polynomial multiplication unit (NTT), which is the major bottleneck in the Kyber scheme. As a first step, an optimized modular multiplication architecture combining KRED and lookup table-based algorithms is presented, which reduces the resources of slices by 16.7%. It is used in a pipelined NTT/INTT architecture that is completely BRAM free and instead uses 3 FIFOs for coefficients storage. We hereby present the most compact FPGA based design for NTT architecture in Kyber till date. Experimental results bench marked on comparable FPGA devices show that our proposed design is 36-75% better than the state-of-the-art implementations in terms of hardware efficiency for NTT/INTT calculations and 3.4-4.4× better for the Point-wise Multiplication (PWM) operation.

Index Terms—Post-quantum cryptography (PQC), Lattice-based Cryptography (LBC), CRYSTALS-Kyber, Lightweight design, FPGA, Number theoretic transform (NTT), Polynomial Multiplication, Hardware.

I. INTRODUCTION

Post-quantum Cryptography (PQC) refers to the class of public-key cryptosystems that can resist attack from quantum computers. PQC is an active area of research since a quantum computer will be able to implement quantum algorithms (Shor's algorithm [1], Grover's algorithm [2]) that can reduce the security strength of all the currently deployed Public Key Encryption (PKE) (including RSA [3], Elliptic Curve Cryptography (ECC) [4]) to 0 bits. In 2016, the PQC standardization process initiated by the National Institute of Standards and Technology (called the NIST PQC) that received 69 cryptographic algorithms in Round 1. In July 2022, NIST PQC announced their first four 'winner' schemes for standardization, with CRYSTALS-Kyber (called Kyber hereafter) [5] being the only PKE/Key Encapsulation Mechanism (KEM) scheme.

Kyber is a lattice-based cryptography scheme whose hardness relies on the Module-Learning with errors (M-LWE) problem. Schemes based on the M-LWE generally have

higher security than Ring-LWE (R-LWE) [6] schemes and can achieve higher performance than LWE schemes [7]. In M-LWE schemes, polynomials are usually defined over the ring $\mathbb{Z}_q[x]/(x^n + 1)$ and using the parameter k to specify the size of the public key matrix and hence the security levels. In Kyber KEM, k is chosen as 2, 3, and 4 to correspond to security levels 1, 3, and 5, as defined by NIST [8]. The structure of Kyber's primes is $q = k \cdot 2^m + 1$, which can perform Number Theoretic Transform (NTT) to accelerate the core computation, polynomial multiplication. After the NIST PQC Round 2, the Kyber team reduced the primes from 7,681 to 3,329, further improving the performances of Kyber.

Modular reduction and the NTT computation are core modules in Kyber hardware design and affect its overall operating frequency and the number of computational cycles. Much work has been reported on efficient hardware modular reduction in Kyber, including recently reported [9]–[11]. In 2021, Xing *et al.* used the Barrett reduction algorithm for partitioning the product of multiplications to compute the modular reduction [9], but the implementation results in larger resources. Zhang *et al.* proposed a new fast modular reduction based on lookup tables [10]. Bisheh-Niasar *et al.* proposed the K2-RED algorithm based on KRED algorithm [12], which computes twice $r \equiv k \cdot c \bmod q$ for the product of multiplication c , reducing c to 12 bits [11]. Some noticeable recent hardware implementations of Kyber-configured NTTs include [10], [11], [13], [14]. In 2021, Bisheh-Niasar *et al.* proposed a 2×2 butterfly unit in NTT architecture, which uses the K2-RED reduction algorithm [11]. Yarman *et al.* designed three different hardware architectures (lightweight, balanced, and high performance) to execute NTT/Inverse-NTT (INTT) using 1, 4, and 16 parallel butterfly units, respectively [13]. In 2022, a new fully pipelined ultra-fast NTT architecture using the Multipath Delay Commutator (MDC) architecture was proposed that had large resource consumption [14].

We present a compact FPGA-based hardware architecture for the NTT operation in CRYSTALS Kyber. As a first step, we undertake the lookup table-based algorithm [10] and the K2-RED algorithm [11] to propose a low area, fast modular reduction algorithm and architecture. In addition, a BRAM-free iterative NTT/INTT architecture is designed using the rearranged Decimation in Time (DIT)/Decimation in Frequency (DIF) strategy as used in [15], replacing all BRAM units with 3 smaller FIFOs. Compared to the state-of-the-art, we

This work is grants from the Engineering and Physical Sciences Research Council (EPSRC) Quantum Communications Hub (EP/T001011/1), and supported by the National Natural Science Foundation of China (62022041) and the Fundamental Research Funds for the Central Universities (NP2022103).

report a significant reduction in area consumption due to the elimination of the BRAMs and is competitive with comparable state-of-the-art in terms of speed, making it suitable for area-constrained applications, like IoT edge nodes.

II. PRELIMINARIES

A. Kyber

Kyber KEM is the only PKE/KEM algorithm that is chosen to be standardized after Round 3 NIST PQC standardization process [16]. Kyber KEM consists of three functions: Key Generation, Encapsulation, and Decapsulation. Kyber KEM is based on the M-LWE mathematical problem and therefore, like other LBC schemes, the main computational bottle neck in its operation is the polynomial multiplication. For example, during Key Generation $\mathbf{t} = \mathbf{s} \circ \mathbf{A} + \mathbf{e}$ is computed, where $\mathbf{A}$ is $k \times k$ -dimensional matrix and public key $pk = \mathbf{t}$, secret key $sk = \mathbf{s}$. The parameter setting of Kyber KEM ($q = 3,329$) ensures that it can be accelerated for polynomial multiplications using the NTT algorithm. In Kyber, a single polynomial contains $n = 256$ coefficients. For more details on Kyber KEM, the reader is kindly referred to [5].

B. NTT in Kyber

The NTT is used in lattice based schemes to accelerate the polynomial multiplication operation. In kyber, for prime $q = 3,329$ with $q - 1 = 2^8 \cdot 13$, the base field $\mathbb{Z}_q$ contains primitive 256-th roots of unity. $X^{256} + 1$ is defined as 128 polynomials of degree 2 modulo q . Given a polynomial in Kyber is $f = \{f_0 + f_1x + \dots + f_{255}x^{255}\} \in R_q$ and ζ is the primitive root of unity modulo q , then the NTT operation can be defined as:

$$NTT(f) = \hat{f} = \hat{f}_0 + \dots + \hat{f}_{255}x^{255} \quad (1)$$

with

$$\hat{f}_{2i+k} = \sum_{j=0}^{127} f_{2i+k} \zeta^{(2\mathbf{br}_7 i+1)j} \quad (2)$$

where $\mathbf{br}_7$ means bit reversal of the unsigned 7-bit integer i and $k \in \{0, 1\}$.

Thus, in Kyber, a single 256 coefficients polynomial can be separated into two 128 coefficients polynomials by parity and the NTT/INTT is independently computed on each. In addition, the NTT output data is ordered in bit reversed addressing and the Point-wise Multiplication (PWM) is also carried out on this bit-reversed ordered samples in Kyber. After INTT, the samples ordering is restored to natural ordering. The optimized PWM is shown as follows [9].

$$\begin{aligned} \hat{h}_{2i} + \hat{h}_{2i+1}x &= \hat{f}_{2i}\hat{g}_{2i} + \zeta^{2\mathbf{br}_7 i+1} \hat{f}_{2i+1}\hat{g}_{2i+1} \\ &+ x(\hat{f}_{2i}\hat{g}_{2i+1} + \hat{f}_{2i+1}\hat{g}_{2i}) \end{aligned} \quad (3)$$

III. PROPOSED BRAM-FREE NTT/INTT ARCHITECTURE

A. Proposed Modular Reduction Architecture

The Modular multiplication operation is critical for an optimized implementation of the NTT/INTT. The modular multiplication proposed in [11] achieves a good throughput

performance, however, using $k \cdot c_l - c_h$ again in the second reduction increases the resource consumption. Therefore, the algorithm in [11] is modified by introduction of lookup tables in [10]. The reduction result of the higher 4 bits of the multiplication result is pre-computed and stored in the FPGA, thus the result of a single round of reduction can be 12-bit. The proposed algorithm is shown in Algorithm 1. The sizes of $k \cdot C_l$, C_h and x are all 12 bits. Thus the result of C' needs only a simple correction to obtain a 12-bit result.

Algorithm 1 The proposed modular reduction algorithm

- 1: **Input:** Multiplication result: $C = \{c_{23}, \dots, c_0\}$,
 $q = 3329, k = 13$.
- 2: **Output:** $C' = -k \cdot C \bmod q$.
- 3: $C_l = (c_7, \dots, c_0)_2$
- 4: $C_h = (c_{19}, \dots, c_8)_2$
- 5: $x = LUT((c_{23}, \dots, c_{20})_2)$
- 6: $C' = x + C_h - k \cdot C_l$
- 7: **return** C'

A fast modular multiplication architecture based on Algorithm 1 is proposed, which uses only two cycles to complete a single reduction and supports pipelining. As shown in Fig. 1, in the first cycle, the four most significant bits (MSBs) x_1 of the multiplication result are fed into the lookup table, and the precomputed data $x = x_1 \cdot 2^{20} \bmod q$ is taken from the lookup table. The $C_h - k \cdot C_l$ can also be calculated in the first cycle, where splitting $k = 13 = 4'b1011$ is done using shift and addition to reduce resource consumption. In the second cycle, the lookup table result x is added to $C_h - k \cdot C_l$ and the result of the addition is extended to 14 bits (The 14th bit is used to represent the sign bit). The data result is then corrected for the higher two bits of the addition result, 12'*hd01* (q) and 13'*h12ff* ($-q$) are used for correction of the positive and negative data, respectively. Compared to [11], 1 addition operation is reduced, and compared to [10], 2 lookup tables, 1 addition and 1 carry-save adder (CSA) addition operation are reduced in our design.

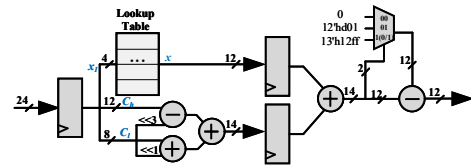


Fig. 1. The proposed low area consumption modular reduction architecture.

B. A BRAM-free Iterative NTT Architecture

Most of the current Kyber hardware implementations use an iterative NTT design in order to reduce the area consumption, i.e., 1, 2, and 4 parallel butterfly units with memory units to complete the NTT computation [10], [11], [13], [14]. In most of these implementations, since multiple intermediate data samples are accessed during the NTT/INTT computation, generally dual-ported BRAMs are used to hold data. While BRAMs can access any address within the memory unit,

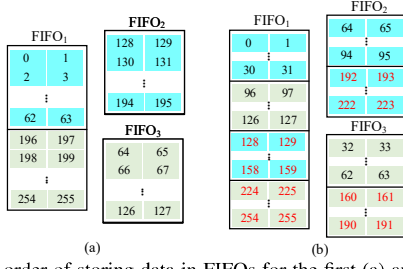


Fig. 2. The order of storing data in FIFOs for the first (a) and second rounds (b) of NTT calculation.

TABLE I
PWM CALCULATIONS IN THE TWO BUTTERFLY UNITS

	First Round	Second Round
Butterfly Unit 1	$s_{2i} = f_{2i} + f_{2i+1}$ $m_{2i} = f_{2i} \cdot g_{2i}$	$h_{2i+1} = s_{2i} \cdot s_{2i+1}$ $- (m_{2i} + m_{2i+1})$
Butterfly Unit 2	$s_{2i+1} = g_{2i} + g_{2i+1}$ $m_{2i+1} = f_{2i+1} \cdot g_{2i+1}$	$h_{2i} = m_{2i+1} \cdot \zeta + m_{2i}$

they consume huge area resources compared to other on-chip storage units, such as LUTRAM. In this paper, the input and output data of single storage within the iterative NTT/INTT is re-ordered to completely replace the use of any BRAM units in the NTT/INTT computations using 3 small FIFOs.

In order to improve the storage efficiency of NTT/INTT, the rearranged iterative DIT NTT and DIF INTT in [15] are used (thus, a single NTT/INTT calculation consists of seven stages/rounds). The combined NTT/INTT butterfly unit of the proposed architecture comprises of two adders, two subtractors, a single DSP composed for multiplication, a modular reduction unit, and a divide 2 (*Div2*) unit. The *Div2* unit can be implemented directly using a combination of selectors and adders. In the proposed architecture, the NTT and INTT use the Cooley–Tukey (CT) and Gentleman-Sande (GS) butterfly units, respectively, to satisfy the correct ordering of the data (natural ordering for DIT NTT input and bit-reversed ordering for the DIF INTT output). Rotation factors ζ required for NTT/INTT are stored in two distributed ROMs with a single ROM size of 12×128 . The rotation factor of NTT is $-k^{-1} \cdot \zeta \bmod q$ to eliminate the effect of $-k$ in the result of Algorithm 1. When computing the INTT, the operation of dividing by 2 is combined into the rotation factor, thus the rotation factor stored is $-k^{-1} \cdot \lfloor (q+1)/2 \rfloor \cdot \zeta \bmod q$. In the Radix-2 NTT/INTT, a single butterfly unit is used to compute a pair of coefficients in a single round, whereas in our architecture, two butterfly units are used, for both odd and even data blocks, as the odd and even data can be computed independently at the same time. For an optimized PWM architecture, the Eq. 3 is divided into two groups and two butterfly units are arranged for computation. As shown in Table I, a single PWM calculation computes two coefficients of each of the two polynomials, and in the second round, the two butterfly units complete the calculation of $s_{2i}, s_{2i+1}, m_{2i}, m_{2i+1}$ using the adder and multiplier, respectively, and store the calculation results in the FIFO. After 128 clock cycles, $s_{2i}, s_{2i+1}, m_{2i}, m_{2i+1}$ is removed from the FIFO and the equation in the second round

in Table I is calculated. Note that the required computation in the second round is one more addition computation than can be carried in the butterfly cell, thus an additional adder is needed. The output in the second round is taken directly as the result of the PWM calculation. Therefore, a total of 265 cycles are required to perform PWM calculations for two polynomials of every 256 coefficients.

C. Memory Approach in the BRAM-free Iterative NTT

Consider a polynomial f with natural ordering of the coefficients is $\{0, 1, 2, 3, \dots, 254, 255\}$. The distance/ gap between the pair of accessed indexes during the first round of NTT calculation is $n/2 = 256/2 = 128$, therefore for the first cycle the order of the input coefficients is $\{(0, 128), (1, 129)\}$, while the $(0, 128)$ representing the indexing order of the input data for one of the butterfly units. As shown in Fig. 2, FIFO₁, FIFO₂, and FIFO₃ are used to store the data output from two butterfly units, where FIFO₁ and FIFO₂ are both 24×128 FIFOs and FIFO₃ is a 24×32 FIFO. The output of the double butterfly units are first stored in FIFO₁ and FIFO₂, as shown in the blue in Fig. 2(a). When the order of the output data is $\{(64, 196), (65, 197)\}$, the FIFO changes the storage approach and stores $\{(64, 65)\}$ into FIFO₃, while $(196, 197)$ is stored in FIFO₁ until FIFO₁ is full and a single round of NTT calculation is complete. When the second round of NTT calculation starts, the data is first taken out from FIFO₁ and FIFO₃ to form the order $\{(0, 64), (1, 65)\}$, and when FIFO₃ is completely read, the data is taken out from FIFO₁ and FIFO₂, thus the output data fully satisfies the order of the second round of input in the NTT.

Fig. 2(b) shows the storage state of the FIFOs after the second round of NTT calculation, where the numbers are the coefficients index. Since the interval of coefficients in the input sequence of the 3^{rd} round of NTT calculation is $n/2^3 = 32$, the storage state changes once after every 64 coefficients (16 cycles). For the first 16 cycles, FIFO₁ and FIFO₂ store the data while in the next 16 subsequent cycles FIFO₁ and FIFO₃ store the data. Afterwards, FIFO₁ and FIFO₂ are used again to store the data (red numbers in Fig. 2(b)), and the storage is exchanged until the FIFO₁ is full. After that, between the $3^{rd} - 6^{th}$ round of the NTT calculation, the three FIFOs alternately store the data; the cycles of storage state transition are gradually shortened to $\{8, 4, 2, 1\}$, until the completion of the seventh round of NTT calculation, the data is directly output. In this case, the output order of the NTT is $\{\mathbf{br}(0), \mathbf{br}(1), \dots, \mathbf{br}(255)\}$, which can be directly used for PWM.

For PWM, when performing the first round of calculations, a single cycle is required to store four intermediate results (48-bit) in FIFO₁ and FIFO₂ and a complete storage of 128 cycles. In the second round of calculation, the butterfly unit gets input from FIFO₁ and FIFO₂.

The computation order of INTT is the opposite to that of the NTT. Fig. 3(a) and (b) represent the order of the data stored in the FIFO for the first and second rounds of INTT computation, respectively. Note that the input order of the INTT is $\{\mathbf{br}(0), \mathbf{br}(1), \dots, \mathbf{br}(255)\}$, thus the numbers in Fig. 3 all represent

TABLE III
RESULTS AND COMPARISON OF NTT/INTT/PWM IMPLEMENTED ON FPGA (ARTIX-7)

Designs	#LUTs	#FFs	#DSPs	#BRAMs	#Slices*	ENS**	Freq. (MHz)	Cycles (NTT/INTT/PWM)	Times (μ s)	A $\times$ T (ENS $\cdot$ ms)
Zhang <i>et al.</i> [10]	609	640	2	4	153	1,137	257	490/490	1.9/1.9/-	2.16/2.16/-
Bisheh-Niasar <i>et al.</i> [11]	810	717	4	2	203	995	222	324/324/-	1.5/1.5/-	1.49/1.49/-
Xing <i>et al.</i> [9]	1,737	1,167	2	3	435	1,223	161	512/576/256	3.2/3.6/1.59	3.91/4.40/1.94
Yarman <i>et al.</i> [13]	948	325	1	2.5	237	827	190	904/904/647	4.8/4.8/3.4	3.97/3.97/2.81
Yarman <i>et al.</i> [13]	2,543	792	4	9	636	2,800	182	232/233/167	1.3/1.3/0.9	3.64/3.64/2.52
Yarman <i>et al.</i> [13]	9,508	2,684	16	35	2377	10,837	172	69/71/47	0.4/0.4/0.3	4.33/4.33/2.96
Ours	1,154	1,031	2	0	445	645	300	456/456/265	1.5/1.5/0.88	0.98/0.98/0.57

* The slice of Ref [9]–[11], [13] is estimated by formula $\#LUTs/4$ as one slice consists 4 LUTs in Artix-7 [17].

** ENS (Equivalent Number of Slices) = $\#Slices + \#DSPs \times 100 + \#BRAMs \times 196$ [14].

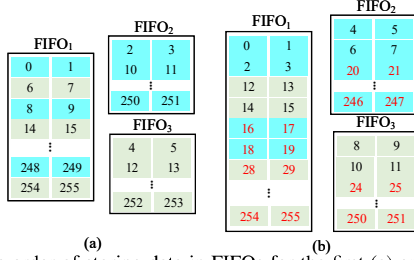


Fig. 3. The order of storing data in FIFOs for the first (a) and second rounds (b) of INTT calculation (Numbers i represent $br(i)$).

the $br(i)$ index position. The cycles of FIFO storage state transitions in the INTT gradually increase, from the first to the sixth round as $\{1, 2, 4, 8, 16, 32\}$. Reading data from the FIFO alternatively continues between reading FIFO₁ and FIFO₃ first, then reading data from FIFO₁ and FIFO₂. The order of data storage is reversed, with data stored into FIFO₁ and FIFO₂ first, then into FIFO₁ and FIFO₃, alternating after the interval cycles for a round. The order of INTT outputs is $\{(0, 128, 1, 129), (2, 130, 3, 131), \dots, (126, 254, 127, 255)\}$.

IV. HARDWARE IMPLEMENTATIONS AND RESULTS

The proposed architectures were implemented and synthesized using Xilinx Vivado 2020.1 Suite on the Artix-7 XC7A200 device. Each design was tested and verified through the functional and post-place and route (PAR) simulations.

TABLE II
COMPARISON OF OUR MODULAR REDUCTION UNIT ON XILINX ARTIX-7

Designs	CPD (ns)	#LUTs	#FFs	#Slices	#DSPs	cycles
KRED [12]	1.99	80	47	31	0	-
Barrett-based [9]	-	135	96	-	0	5
K2-RED [11]	0.91	54	30	18	0	2
Ours	1.6	50	34	15	0	2

Table II shows the hardware implementation results and compares the proposed reduction unit architecture with the state-of-the-art. Our proposed reduction unit outperforms in terms of both resources and critical path delay (CPD) compared to the KRED [12] and Barrett-based [9]. Compared to the K2-RED algorithm [11], our design is more compact with 16.7% lower slice usage but lags behind the K2-RED Critical Path Delay (CPD). However, since the CPD of the complete Kyber architecture is not determined by the reduction unit, its CPD does not lower the overall design performance.

Table III shows the hardware implementation results and comparisons of our NTT/INTT/PWM unit. Our proposed BRAM-free architecture consumes less resources than the state-of-the-art iterative NTT/INTT/PWM architectures since a single BRAM is equivalent to 196 slices [14], with a single slice comprising of 4 LUTs in the Xilinx FPGA [17]. Compared to Zhang *et al.* design [10], our proposed architecture is 21% faster and eliminates the use of 4 #BRAMs. Our proposed design is comparable to [11] in terms of speed and #LUTs consumption, but requires fewer number of #DSPs and #BRAMs. In addition, our proposed architecture support PWM calculations as well. Compared with [9], our proposed design exhibits 53.1% speedup for NTT, 65.6% for INTT and 44.7% for PWM calculations and reduces 47.2% ENS. Compared to Yarman *et al.* design [13], our proposed BRAM-free design has 68.8% speedup for NTT/INTT and 74.1% for PWM computations over their single butterfly unit design and a 22% reduction on ENS; for their NTT/INTT design with 4, 16 butterfly units. Our architecture substantially reduces the use of #LUTs and #BRAMs and has better hardware efficiency. In terms of A $\times$ T (area time product), the proposed design reduces 54.6%, 36.4%, 74.9%, 73.1% on NTT/INTT compared to [10], [11], [9], [13] (Optimal A $\times$ T design) and reduces 3.4 and 4.4 $\times$ on PWM compared to [9], [13], respectively.

V. CONCLUSION

This work presents an ultra-lightweight BRAM-free iterative NTT based CRYSTALS-kyber compatible polynomial multiplication architecture that utilizes three FIFOs with different storage orders, eliminating the BRAMs needed for conventional iterative NTT designs and saving a large amount of on-chip storage resources. Moreover, the proposed architecture is comparable in speed to the state-of-the-art designs for NTT/INTT and 44.7% faster for the Point-wise Multiplication (PWM). The proposed architecture also improves hardware efficiency by 36-75% for the NTT/INTT and 3.4-4.4 $\times$ for the PWM computation as compared to the current state-of-the-art architectures. In addition, this work proposes a reduction unit algorithm that combines the KRED algorithm with the lookup tables-based algorithm, consuming lower resources. Our work achieves the smallest hardware footprint for a NTT based polynomial multiplier FPGA implementation reported till date with a substantially better AT efficiency, much needed for the lightweight Kyber designs, practical for the IoT based applications.

REFERENCES

- [1] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM review*, vol. 41, no. 2, pp. 303–332, 1999.
- [2] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
- [3] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Communications of the ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [4] V. S. Miller, "Use of elliptic curves in cryptography," in *Conference on the theory and application of cryptographic techniques*, pp. 417–426, Springer, 1985.
- [5] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber (version 3.0) – submission to round 3 of the nist post-quantum project," *NIST, Tech. Rep.*, 2020.
- [6] V. Lyubashevsky, C. Peikert, and O. Regev, "On ideal lattices and learning with errors over rings," in *Annual international conference on the theory and applications of cryptographic techniques*, pp. 1–23, Springer, 2010.
- [7] R. Lindner and C. Peikert, "Better key sizes (and attacks) for LWE-based encryption," in *Cryptographers' Track at the RSA Conference*, pp. 319–339, Springer, 2011.
- [8] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehlé, "CRYSTALS-Kyber algorithm specifications and supporting documentation," *NIST PQC Round 3*, 2020.
- [9] Y. Xing and S. Li, "A compact hardware implementation of CCA-secure key exchange mechanism CRYSTALS-kyber on FPGA," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 328–356, 2021.
- [10] C. Zhang, D. Liu, X. Liu, X. Zou, G. Niu, B. Liu, and Q. Jiang, "Towards efficient hardware implementation of NTT for kyber on FPGAs," in *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, IEEE, 2021.
- [11] M. Bisheh-Niasar, R. Azarderakhsh, and M. Mozaffari-Kermani, "High-speed NTT-based polynomial multiplication accelerator for post-quantum cryptography," in *2021 IEEE 28th Symposium on Computer Arithmetic (ARITH)*, pp. 94–101, IEEE, 2021.
- [12] D. T. Nguyen, V. B. Dang, and K. Gaj, "High-level synthesis in implementing and benchmarking number theoretic transform in lattice-based post-quantum cryptography using software/hardware codesign," in *International Symposium on Applied Reconfigurable Computing*, pp. 247–257, Springer, 2020.
- [13] F. Yaman, A. C. Mert, E. Öztürk, and E. Savaş, "A hardware accelerator for polynomial multiplication operation of CRYSTALS-Kyber PQC scheme," in *Design, Automation Test in Europe Conference Exhibition (DATE)*, pp. 1020–1025, IEEE, 2021.
- [14] D.-e.-S. Kundi, Y. Zhang, C. Wang, A. Khalid, M. O'Neill, and W. Liu, "Ultra high-speed polynomial multiplications for lattice-based cryptography on FPGAs," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–1, 2022.
- [15] T. Pöppelmann, T. Oder, and T. Güneysu, "High-performance ideal lattice-based cryptography on 8-bit ATxmega microcontrollers," in *International conference on cryptology and information security in Latin America*, pp. 346–365, Springer, 2015.
- [16] A. Gorjan, A. Daniel, C. David, D. Quynh, D. Thinh, K. John, L. Jacob, M. Carl, M. Dustin, P. Rene, P. Ray, R. Angela, S.-T. Daniel, and L. Yi-Kai, "Status report on the third round of the NIST post-quantum cryptography standardization process," <https://csrc.nist.gov/publications/detail/nistir/8413/final>, 2022.
- [17] Xilinx, "7 series FPGAs configurable logic block: User guide," *UG474 (v1.8)*, September 27, 2016.